

PROJECT\_ROOT/

src/

main.py

config.json

ai\_agents/

docs/

playbook\_engenharia\_ia.md

architecture\_diagram.svg

tests/

test\_suite.py

playbook\_engenharia\_ia.md

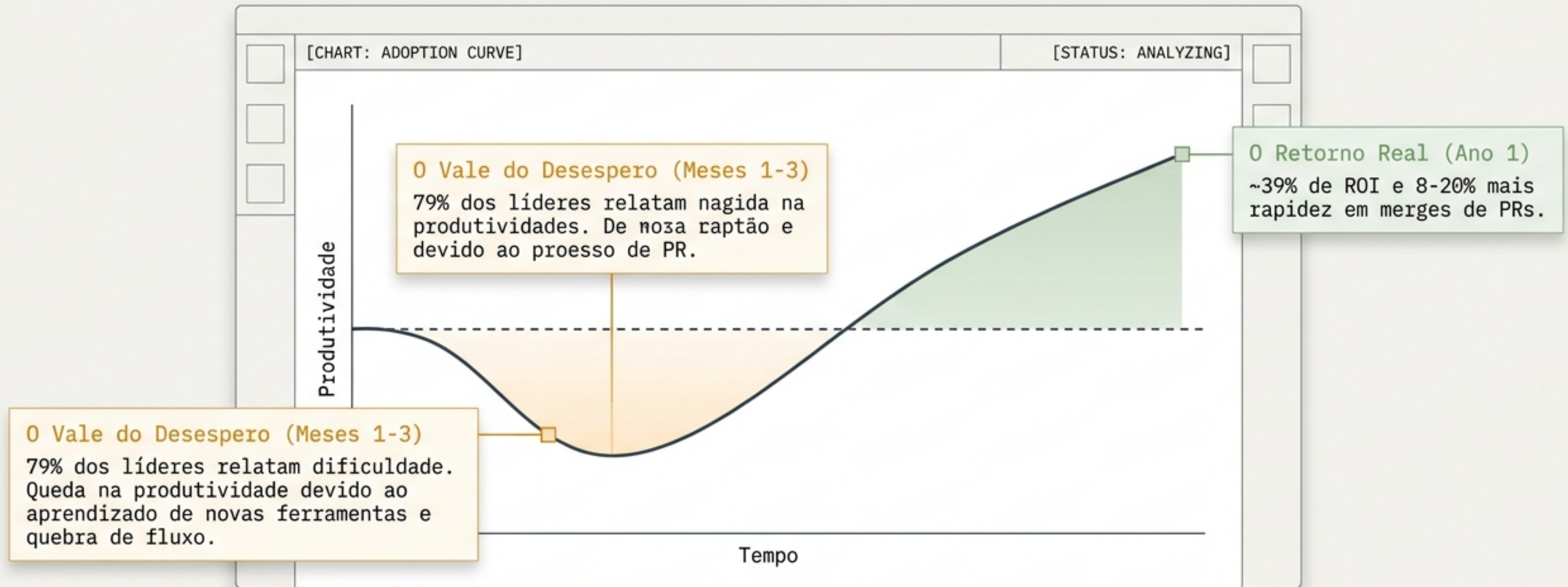
×

# O Terminal do Arquiteto: Desenvolvimento Assistido por IA

Do VibeCoding caótico à orquestração arquitetural. O playbook definitivo para desenvolvedores sêniores dominarem o contexto e construírem software com IA.

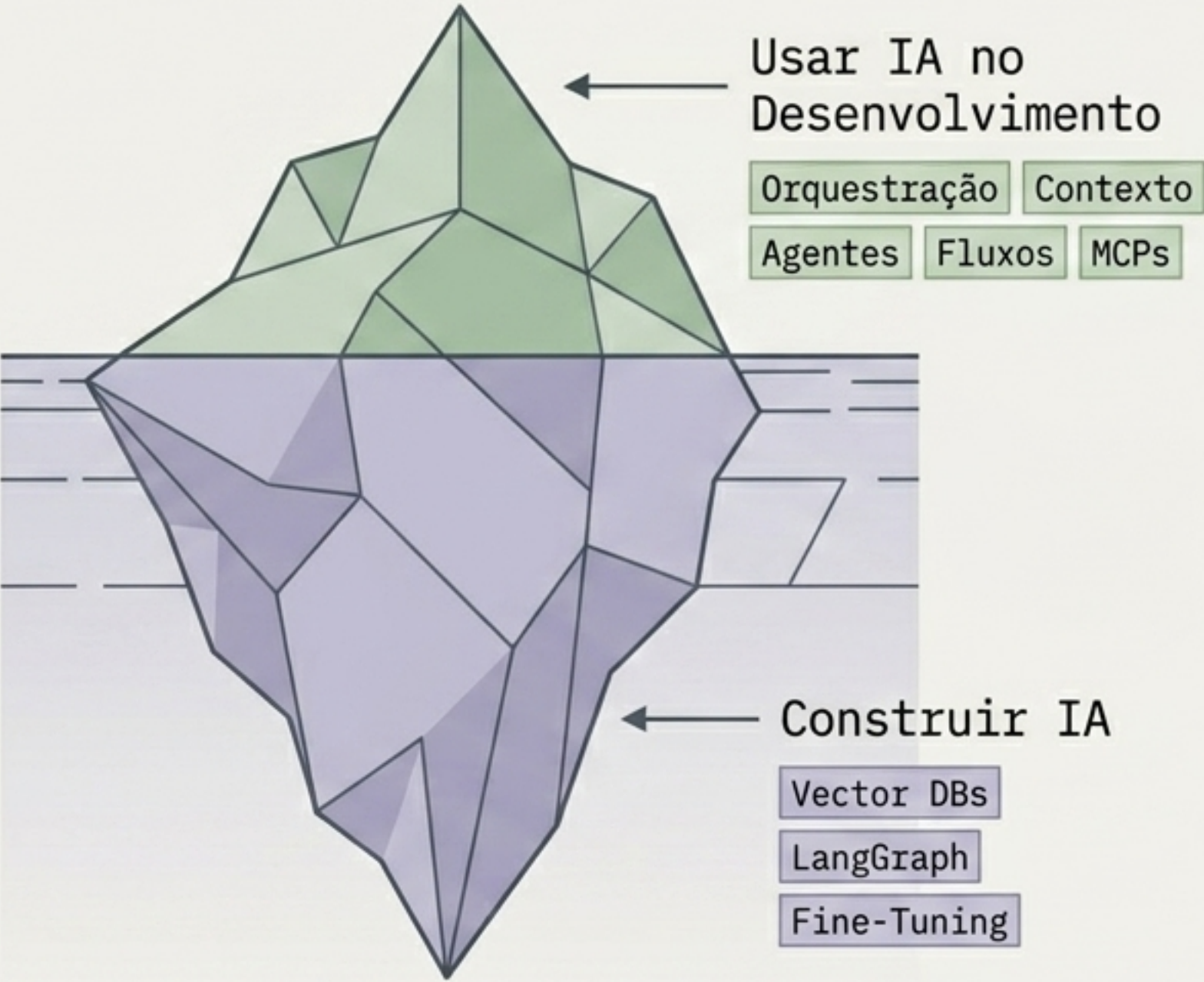
```
> System initialized...  
> Loading Developer Playbook...  
> Status: Ready
```

# O Choque de Realidade: A Curva de Adoção DORA



Esqueça a promessa de produtividade 10x. Se a IA nos der 50% de ganho e melhorar a qualidade de vida do desenvolvedor, já é uma revolução. O segredo não é a mágica, é a cultura de engenharia.

# O Iceberg da IA: Onde Focar Seu Esforço

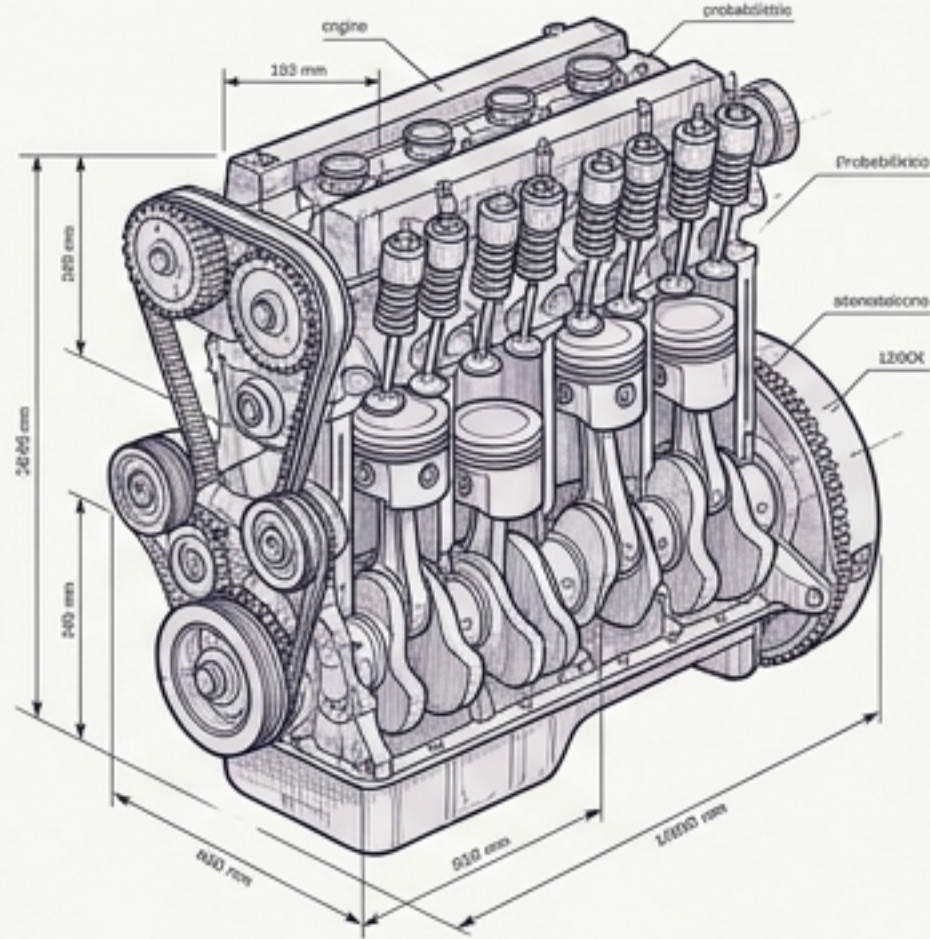


Nota: Foque no topo a menos que haja demanda explícita para o fundo.

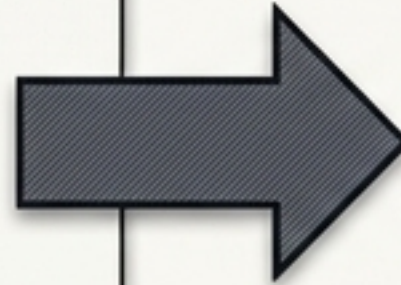
|           | VibeCoding                                           | Desenvolvimento Assistido                          |
|-----------|------------------------------------------------------|----------------------------------------------------|
| Propósito | Protótipos e apps pequenos                           | Lógica de negócio complexa                         |
| Público   | Usuários não-técnicos                                | Engenheiros de Software Sêniores                   |
| Dinâmica  | A IA "cospe" o código e o usuário torce pelo melhor. | O Dev lidera, orchestra o contexto e a IA assiste. |

# 0 Motor vs. 0 Carro (Entendendo o Harness)

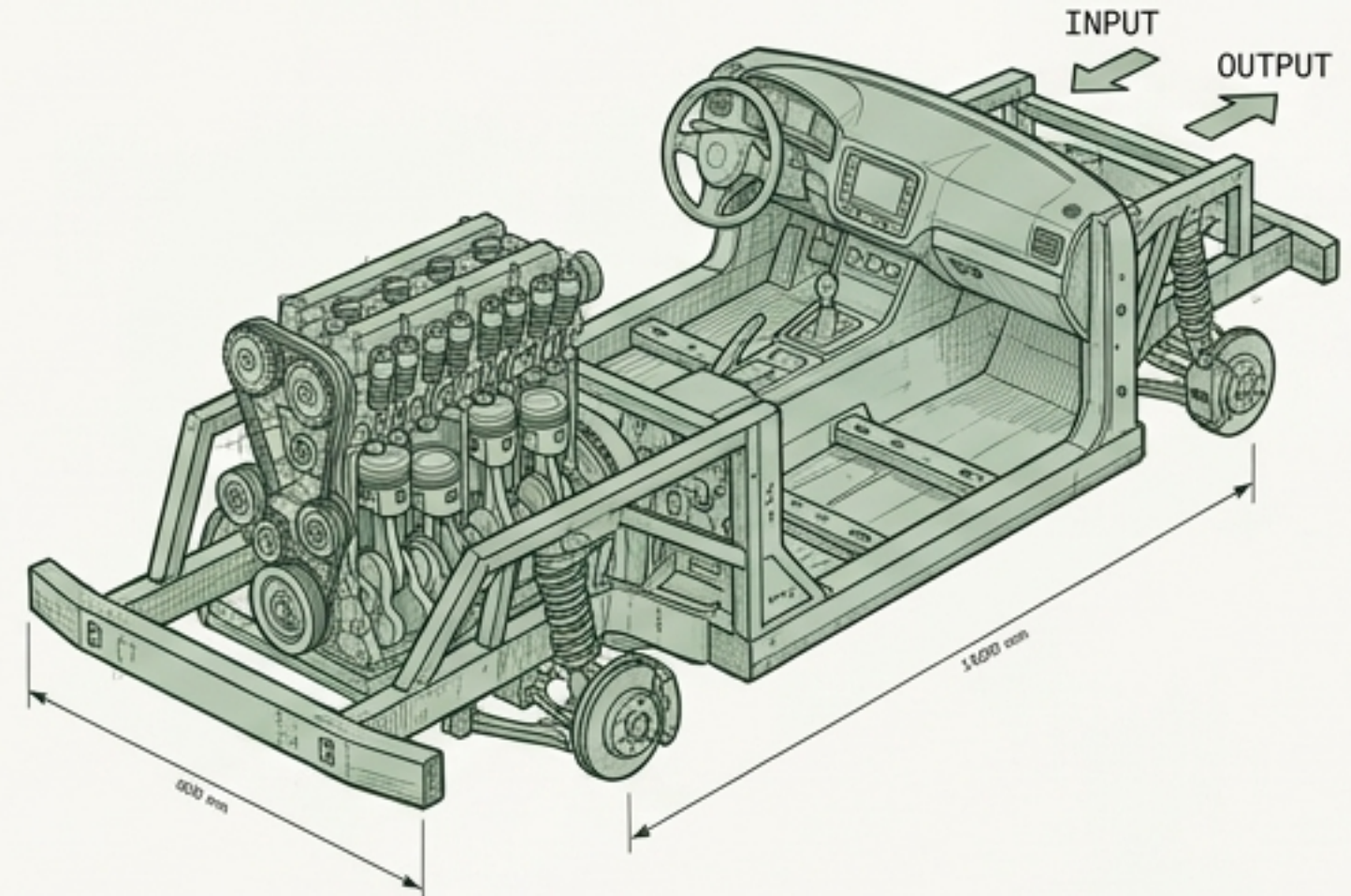
## 0 Motor (LLM)



- Modelo Probabilístico
- Máquina de previsão de próxima palavra
- Não-determinístico e cego sobre o mundo externo



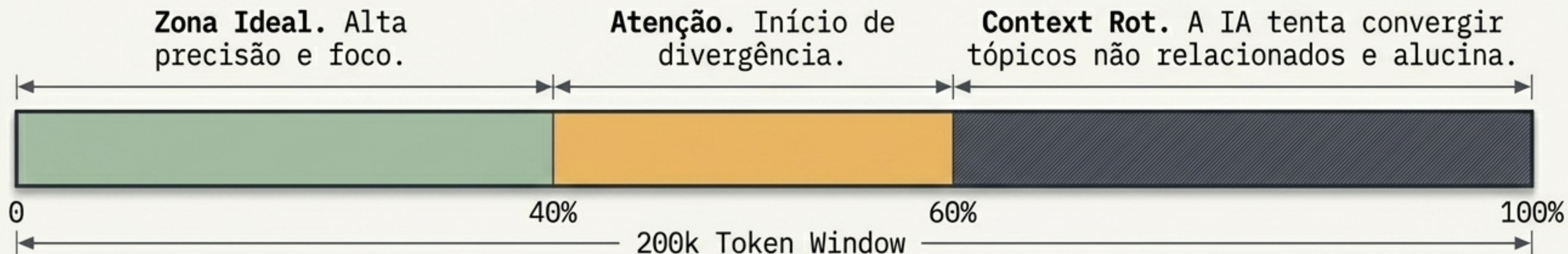
## 0 Carro (0 Agente / Harness)



- Estrutura de Contenção e Orquestração
- Define limites, ferramentas (Tools) e restrições
- Injeta contexto sob demanda

0 segredo do desenvolvimento moderno com IA não está em ter o melhor modelo, mas em **construir o melhor** Harness ao redor dele.

# A Grande Armadilha: Context Rot



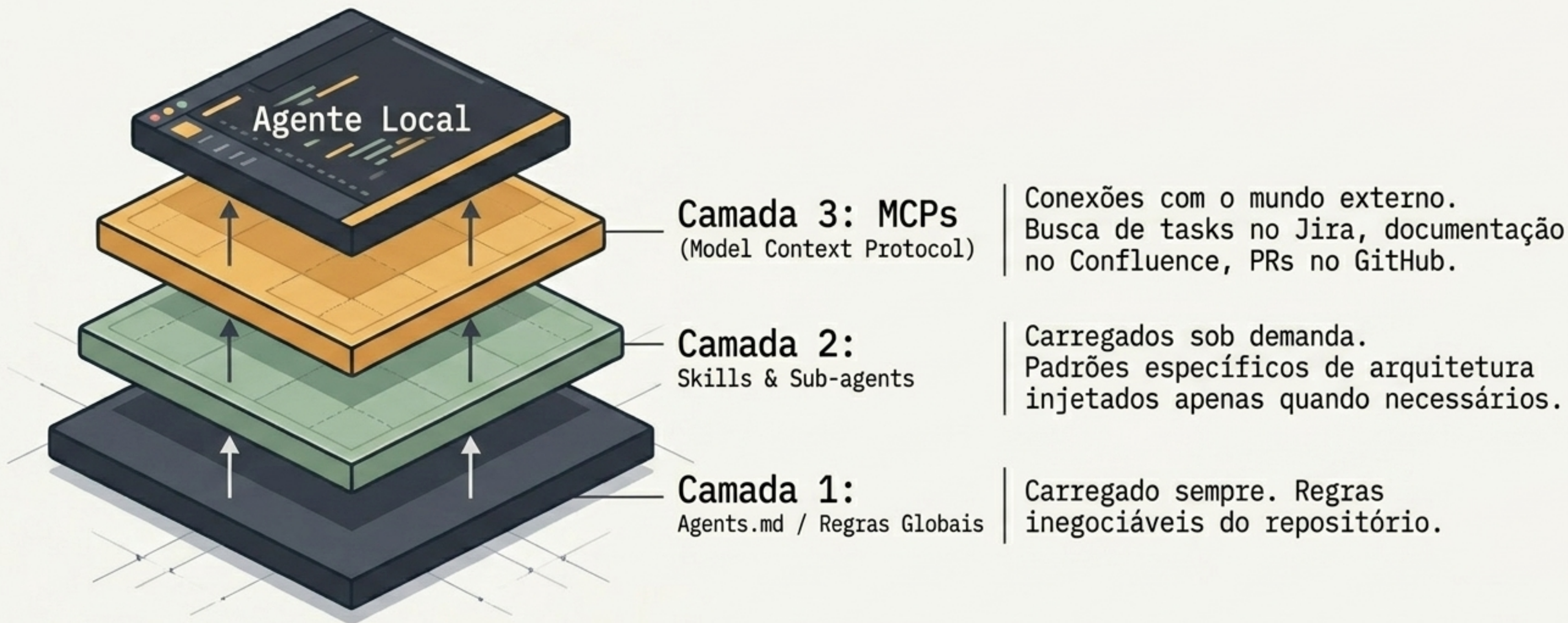
## Problema:

Você usa 17% do contexto para uma task.  
Na mesma janela, você pede algo totalmente diferente. A IA se perde tentando mesclar os dois assuntos e degrada a resposta.

## REGRA DE OURO

Nunca deixe a ferramenta compactar seu contexto. Sempre abra uma **nova janela de agente** ao iniciar um novo fluxo ou task isolada.

# Arquitetura de Contexto: As Camadas do Sistema



A Engenharia de Contexto substitui os prompts gigantes por inferência. Você não precisa enviar arquivos manualmente; o agente entende quando puxar a informação certa.

# Camada 1: O Arquivo Agents.md (Restrições Globais)

agents\_bad.md ×

1 |

❌ O que NÃO fazer

❌ Não crie um arquivo gigante e prolixo.

❌ Não peça para a IA gerar o **Agents.md** para você.

❌ Não use termos vagos como 'otimizar' ou 'melhorar o código'.

agents\_gold\_standard.md ×

1 |

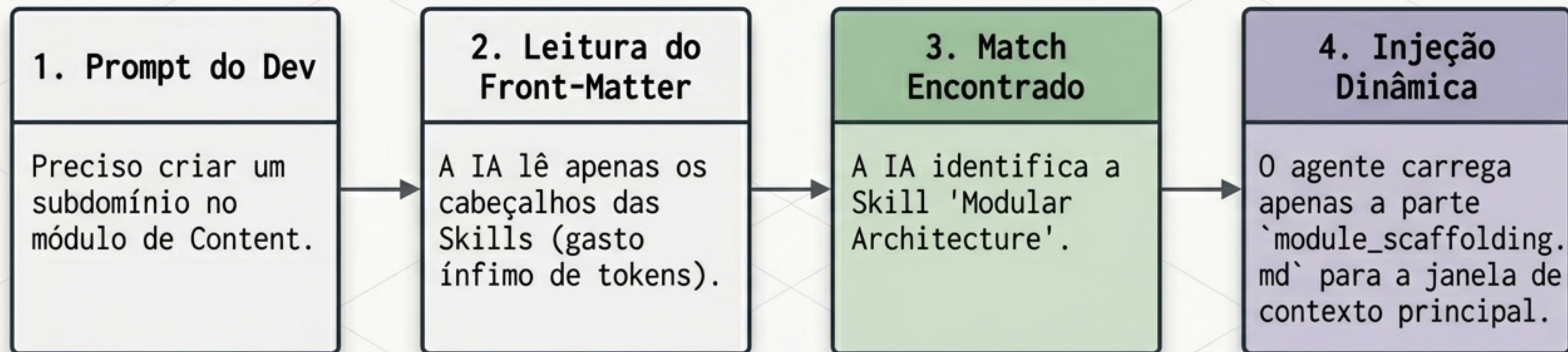
✅ O Padrão Ouro

# Regras Gerais

- Sempre inclua testes cobrindo happy paths e edge cases.
- Se estiver alterando o banco, use docs/database\_patterns.md

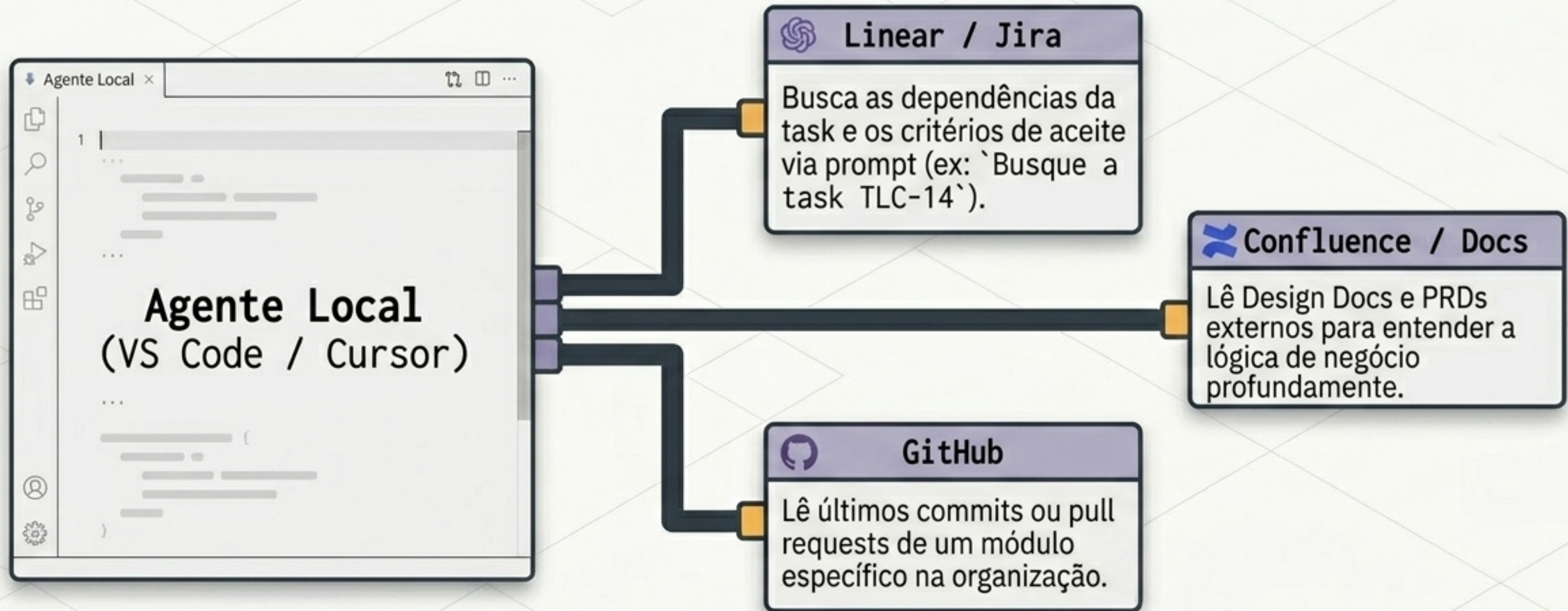
👉 **Dica:** Forneça exemplos reais de código. A IA segue exemplos infinitamente melhor do que instruções teóricas abertas.

## Camada 2: Carregamento Sob Demanda (Skills)



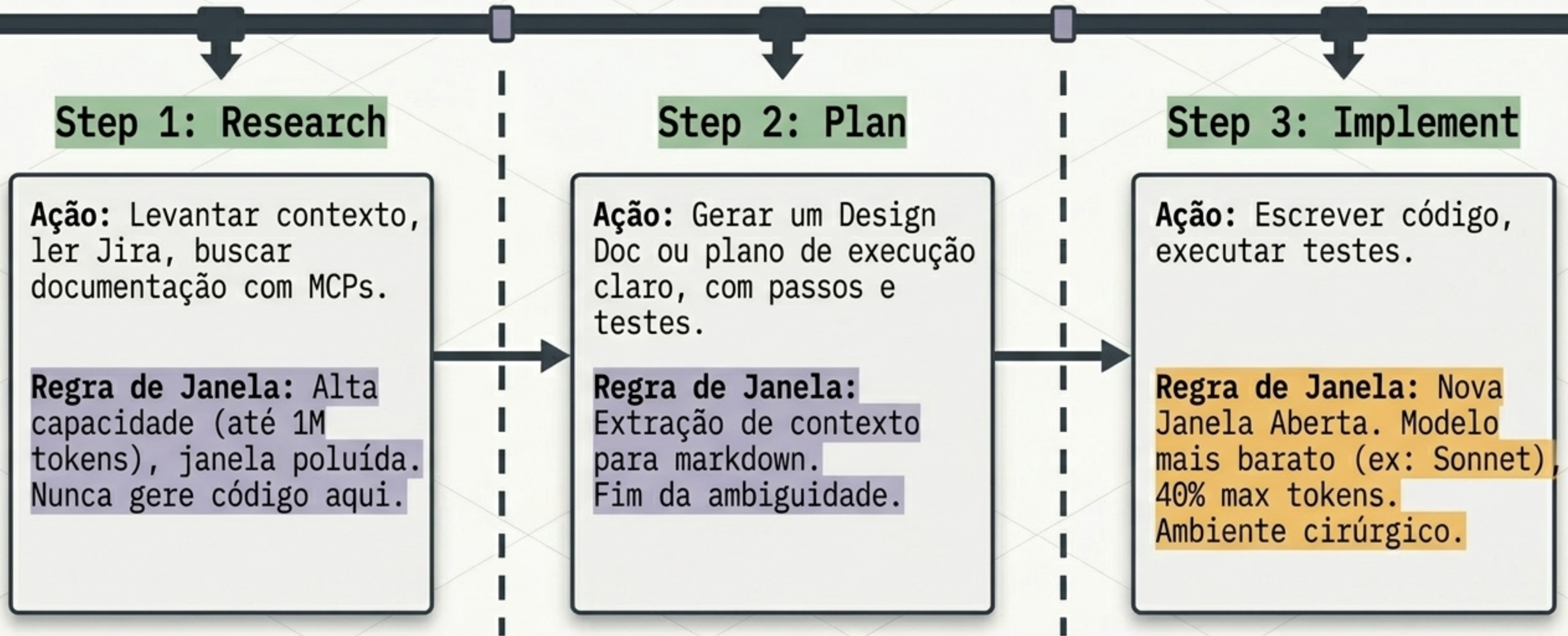
Não tenha um arquivo de arquitetura monolítico com 1000 linhas. Quebre em partes menores e use referências no seu `Agents.md` para que a IA busque apenas o que precisa, otimizando a economia de tokens.

## Camada 3: MCPs e o Mundo Externo



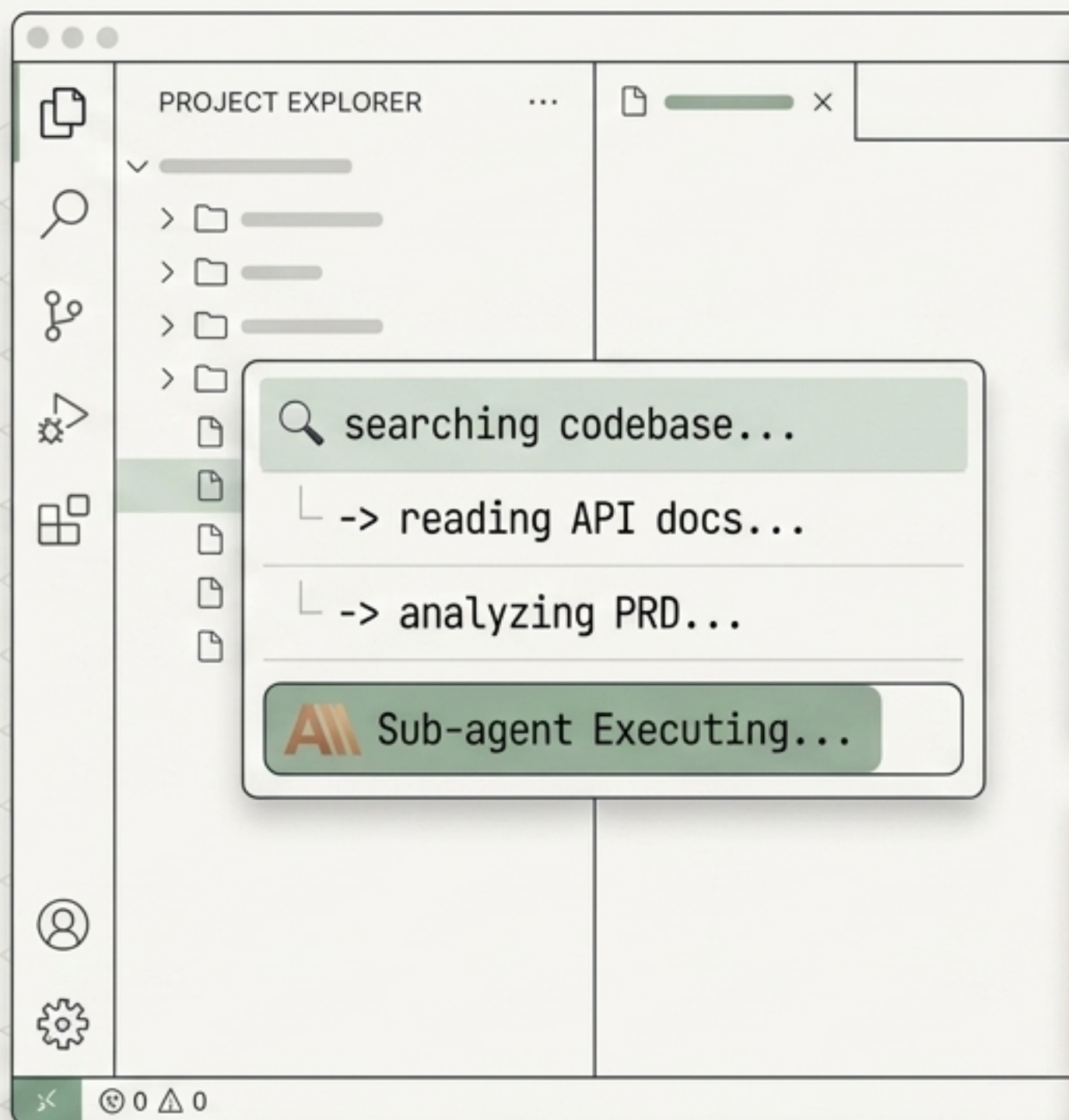
MCP (Model Context Protocol) é um padrão da indústria. Ele lida com OAuth, segurança e tokens fora do seu código base, permitindo que a IA enxergue a empresa inteira sem sair da sua IDE.

# O Workflow Definitivo: O Método R.P.I.



A separação estrita de instâncias é o que previne o Context Rot.

# Fase 1: Research (A Busca e o Discovery)



## A Abordagem

Use modelos densos de raciocínio (como Claude Opus) para analisar PRDs, ler transcrições de reuniões ou pesquisar o codebase inteiro.

## O Papel dos Sub-agents

Ferramentas modernas delegam pesquisas extensas para **Sub-agentes** genéricos. Eles vasculham o código e devolvem apenas um resumo limpo, poupando a janela de contexto principal.

## O Fim da Sessão

**NUNCA** peça para implementar o código na mesma janela onde você passou dias pesquisando. O contexto já está sobrecarregado. Salve as conclusões em **Markdown** e feche a janela imediatamente.

# Fase 2: Plan (Desenvolvimento Spec-Driven)

## O Prompt Humano

Baseado no nosso research, crie um `plano passo a passo` para a `integração` do Stripe.

## O Artefato Gerado (Markdown)

- Fase 1: Foundation e Setup (Credenciais e Client).
- Fase 2: Implementar Mutações e Endpoints.
- Fase 3: Validação e Testes End-to-End (TDD).

Quando você não dá passos claros, a IA toma decisões sob pressão — e faz trade-offs perigosos (como ignorar testes). Um plano escrito e aprovado pelo humano atua como um Trilho impenetrável.

# Fase 3: Implement (Execução Cirúrgica)

## 1. Aba Limpa

Inicie uma aba totalmente nova de agente. O contexto está 100% zerado e livre de poluição de pesquisas passadas.

## 2. Injeção do Plano

Cole o plano `Markdown` exato gerado na Fase 2 (ou invoque a `Skill de Spec`) como o prompt inicial.

## 3. Otimização de Custos

Mude para um modelo mais rápido e barato (ex: `Sonnet`). Ele só precisa seguir e executar o roteiro decidido por um modelo mais 'inteligente'.

## 4. Testes como Guia

A IA implementa a lógica forçada a passar pelos critérios de `TDD` definidos rigidamente no plano, garantindo qualidade previsível.

# O Dilema do Código Legado: Orquestrar antes de Refatorar



**Atenção:** Nunca jogue IA em um código legado cego e peça: "Refatore". O resultado será um desastre.

## 1. Documentar a Realidade

Pergunte à IA quais bizarrices ela vê no código atual. Coloque essas regras literais no **Agents.md**. (ex: 'Neste projeto, a lógica fica no Controller'). Aceite a realidade.

## 2. Estabilizar o Harness

Trabalhe no código seguindo o padrão legado sujo até que a IA consiga realizar tarefas básicas com previsibilidade extrema e sem alucinar arquiteturas ideais.

## 3. Refatorar Gradualmente

Apenas com o sistema estável e o contexto dominado, comece a remover as regras provisórias do **Agents.md** e peça para a IA refatorar módulos gradativamente.

# A Evolução Sênior: O Contexto é o Novo Código

## **A digitação de sintaxe foi commoditizada.**

O valor do Engenheiro não está mais em decorar código, mas em orquestrar sistemas complexos, gerenciar restrições rígidas e comunicar contextos de negócio.

### O Paradigma de Ontem

O Dev operava como um tradutor direto de requisitos soltos em linhas de código isoladas.

### O Paradigma de Hoje

O Dev atua como Arquiteto de Sistemas e Product Engineer, liderando agentes autônomos.

**Domine o **Harness**. Proteja seu contexto. Construa o futuro.**